// THE COROUTINE COURT

I cancelled it. It kept running.

A coroutine I cancelled ran for
8 more seconds. It did not care.

The culprit was one line
I wrote myself.

// THE MENTAL MODEL

Cancel is a message, not a bullet

Cancelling a coroutine throws a CancellationException up through your suspend calls.

That exception IS the message:
we are done here, unwind.

// THE TRAP

The line that ate the message

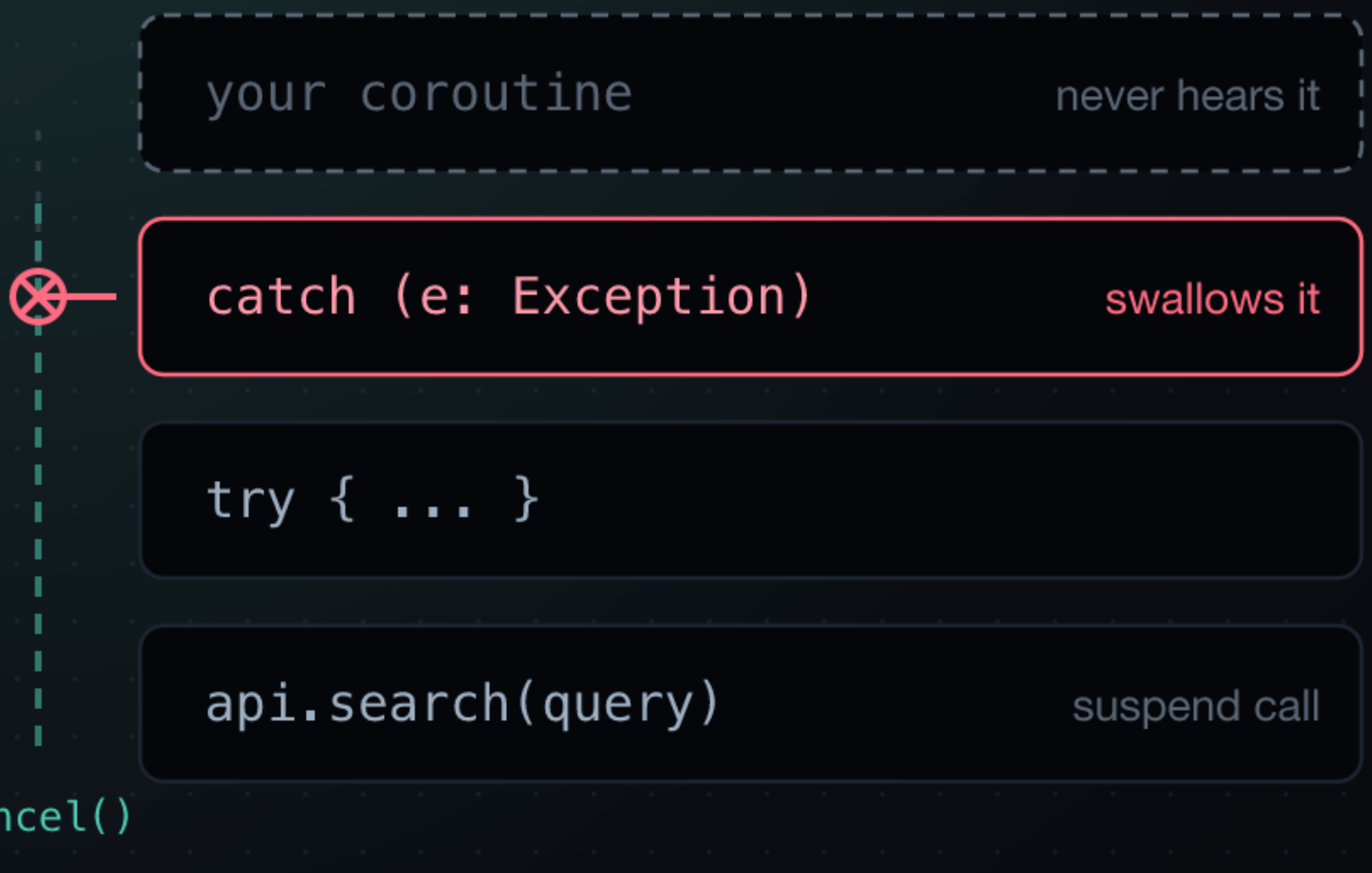
SearchViewModel.kt

```
try {
    results = api.search(query)
} catch (e: Exception) {
    log("search failed")
}
```

CancellationException is an Exception.
So this catch grabs it, logs the wrong thing, and swallows it whole.

// WHAT ACTUALLY HAPPENS

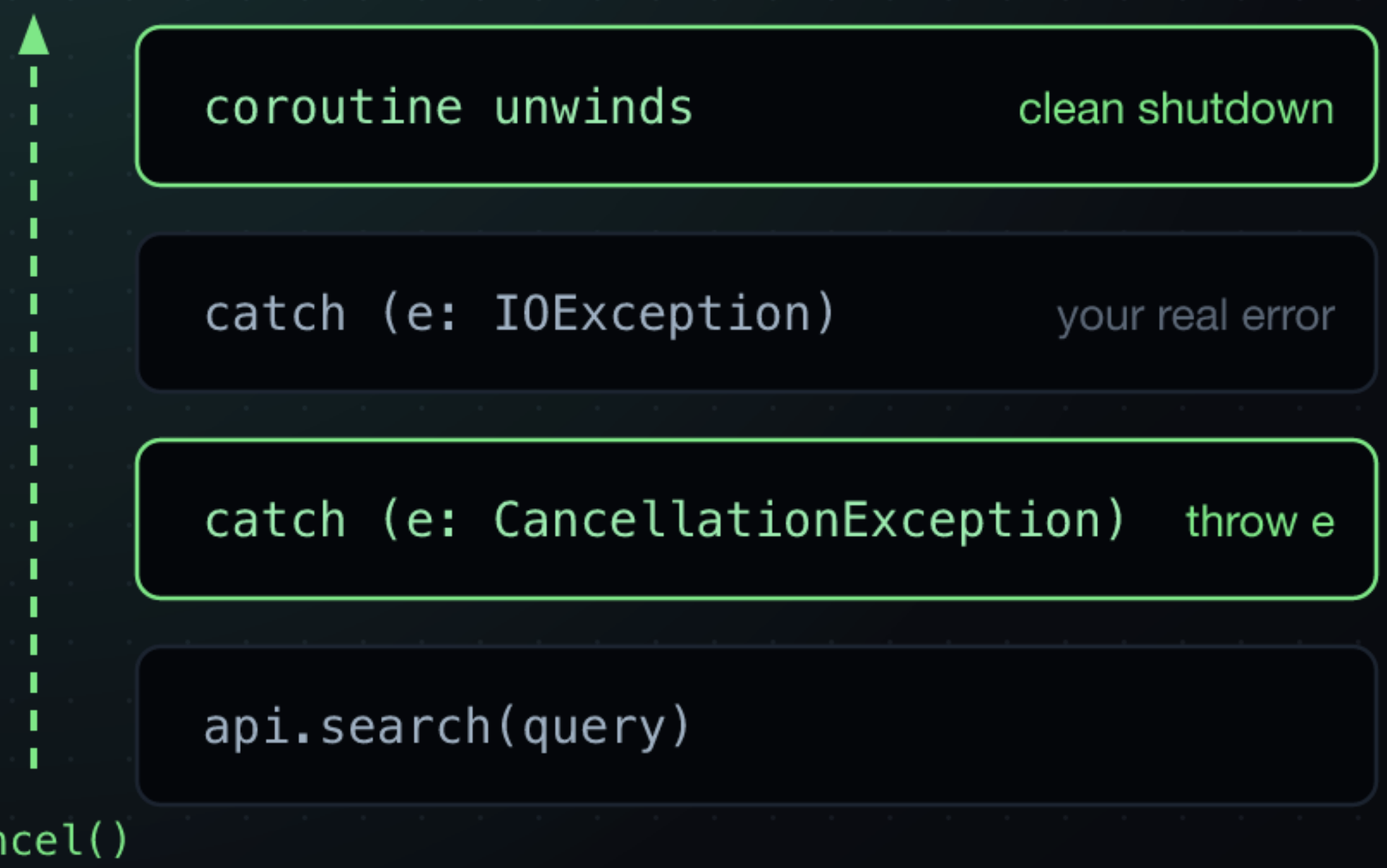
You jailed the messenger



The signal never reaches the coroutine. It keeps running, and sometimes wins the race against the fresh request.

// THE FIX, DRAWN

Let the message through



Rethrow cancellation before your broad catch, or narrow the catch to the exception you actually expect.

// PICK ONE

Four ways out

01 Catch what you expect

`IOException, HttpException. Not Exception.`

02 Rethrow cancellation first

`catch (e: CancellationException) { throw e }`

03 runCatching has the same trap

`it catches CancellationException too`

04 Cleanup that must run

`withContext(NonCancellable) { }`

// THE PAYLOAD

Do not shoot the messenger

Cancellation is a conversation, not a kill switch.

Swallow the message and the work does not stop. It just stops telling you it is still running.